

Bases de datos

UD10.1. Programación en bases de datos

- CONCEPTOS BÁSICOS



1.ÍNDICE

1. INTRODUCCIÓN
2. CONCEPTOS BÁSICOS
3. ESTRUCTURAS DE CONTROL CONDICIONALES
4. ESTRUCTURAS DE CONTROL ITERATIVAS
5. PROCEDIMIENTOS Y FUNCIONES
6. CURSORES
7. TRIGGERS

INTRODUCCIÓN

MySQL permite la programación de funciones, procedimientos almacenados y triggers. Estas herramientas permiten automatizar tareas, mejorar la integridad de los datos y optimizar el rendimiento de las consultas.

Para crear estos objetos se hace uso de un lenguaje de programación que extiende el SQL.

Oracle utiliza PL/SQL (Procedural Language/Structured Query Language) que incluye:

- ▶ Manejo de variables.
- ▶ Estructuras modulares.
- ▶ Estructuras de control de flujo y toma de decisiones.
- ▶ Control de excepciones.

Otros SGBD tienen implementaciones similares. MySQL no tiene ningún lenguaje específico, pero incluye muchas de las características de estos otros lenguajes para la creación de procedimientos y funciones almacenadas.

Los códigos desarrollados en estos lenguajes se pueden almacenar como objetos de la base de datos, creando funciones y procedimientos y reutilizándose por otros usuarios.

BLOQUES DE CÓDIGO

El código más básico en PL/SQL son los **bloques anónimos** que se pueden introducir y ejecutar directamente en la consola SQL.

Ejemplo de bloque anónimo en Oracle:

```
DECLARE
    v_mensaje VARCHAR2 (50) ;
BEGIN
    v_mensaje := 'Hola mundo';
    DBMS_OUTPUT.PUT_LINE (v_mensaje) ;
END ;
/
```

Hemos visto que PL/SQL es propietario de Oracle, por lo que no todos los SGBD puede hacer uso de ellos.

MySql no permite el uso de estos bloques, es necesario asignar un nombre al trozo de código escrito y almacenarlo en la base de datos para poder ejecutarlo.

DELIMITER

DELIMITER es un comando en MySQL que cambia el carácter que indica el fin de una instrucción SQL.

Se usa principalmente cuando se escriben procedimientos almacenados, funciones, triggers o eventos, ya que estos pueden contener múltiples instrucciones SQL separadas por punto y coma (;), lo que puede causar errores de interpretación.

Por defecto, MySQL usa el “;” para terminar una instrucción. Sin embargo, al definir estructuras como BEGIN...END, dentro de procedimientos o triggers, es necesario cambiar temporalmente el delimitador para evitar conflictos con los puntos y comas internos.

Por ejemplo, si ponemos la sentencia: `DELIMITER $$`

A partir de ese momento debemos usar la cadena ‘\$\$’ para ejecutar instrucciones. Así, si queremos ejecutar una consulta que selecciona todos los registros de la tabla productos, pondremos:

```
SELECT * FROM productos$$
```

Si queremos volver al “;”, simplemente volvemos a usar el comando: `DELIMITER ;`

CONCEPTOS BÁSICOS

- ▶ **Bloque:** es un trozo de código que puede ser interpretado por el SGBD. Se delimita con las palabras BEGIN y END.
- ▶ **Programa:** es un conjunto de bloques que realizan una determinada labor.
- ▶ **Procedimiento:** es un programa almacenado en la base de datos y que puede ser ejecutado si se desea invocándolo por su nombre (siempre que se tenga permiso para su acceso). Puede recibir parámetros de entrada, pero no de salida.
- ▶ **Función:** es un programa que bien sea a partir de unos parámetros de entrada o no, devuelve un valor o resultado (datos de salida).

Al igual que los procedimientos, una vez almacenadas, las funciones se pueden utilizar en cualquier expresión desde cualquier otro programa e incluso desde una instrucción SQL.

- ▶ **Trigger** (disparador): es un programa SQL que se ejecuta automáticamente cuando ocurre un determinado suceso a un objeto de la base de datos.

VARIABLES

- ▶ **Variables definidas por el usuario.**

- ▶ Llevan el prefijo '@',
- ▶ Se pueden usar sin declararlas ni inicializarlas. Si no se inicializan por defecto son tipo cadena y tienen valor NULL. Se consultan con la sentencia SELECT:

```
SELECT @nombre_variable;
```

- ▶ Se pueden inicializar usando las palabras SET o SELECT:

```
SET @menor = 10000, @mayor = 50000;  
-----  
SELECT @menor := 10000, @mayor := 50000;
```

- ▶ Ej:

```
SELECT * FROM EMPLEADO
```
- ▶ ..

```
WHERE SALARIO BETWEEN @menor AND @mayor;
```

- ▶ Se pueden ver las variables a partir de la versión de MySQL 8.0.23, usando el comando:

```
SELECT *, CONVERT(VARIABLE_VALUE USING utf8) FROM performance_schema.user_variables_by_thread;
```

VARIABLES

▶ Variables locales.

- ▶ No llevan prefijo.
- ▶ Se declaran con la palabra reservada DECLARE dentro de un bloque de código (BEGIN ...END). Fuera de este bloque **no se pueden usar**.
- ▶ Si no ponemos valor por defecto, éste será NULL.
- ▶ Se les asigna un valor usando SET o SELECT ... INTO :

```
SET variable = '1990-01-01';  
-----  
SELECT SYSDATE() INTO variable;
```

▶ Ej:

```
BEGIN  
DECLARE x INT DEFAULT 0;  
...  
SET X = 10;  
...  
END;
```


VARIABLES

► Variables del sistema.

- Llevan el prefijo '@@'.
- Pueden ser globales (GLOBAL), de sesión (SESSION) o ambas (BOTH).
- Se consultan con la sentencia SELECT o con la instrucción SHOW VARIABLES:

```
SHOW VARIABLES LIKE '%wait_timeout%';  
SELECT @@sort_buffer_size;
```

- Hay muchas y dependiendo de las necesidades, deberemos modificarlas usando la instrucción SET si tenemos los privilegios adecuados (ya hemos hecho esto con AUTOCOMMIT).

Ej:

```
-- Modificar variables GLOBALES:  
SET GLOBAL sort_buffer_size=1000000;  
SET @@global.sort_buffer_size=1000000;  
  
-- Modificar variables de SESIÓN:  
SET sort_buffer_size=1000000;  
SET SESSION sort_buffer_size=1000000;
```

```
[etiqueta_begin:] BEGIN  
[lista de sentencias]  
END [etiqueta_end]
```

BLOQUES

- ▶ Cada programa en está formado por grupos de órdenes SQL llamadas bloques. Cada bloque puede contener, a su vez, nuevos bloques, es decir, permiten anidamiento.

Etiquetar bloques en MySQL sirve para mejorar la claridad, organización y control del flujo de ejecución, sobre todo cuando hay anidamiento. Veremos esto en mayor profundidad cuando veamos el comando LEAVE.

- ▶ Sentencia **DECLARE**. Sólo se puede usar dentro de un bloque y sirve para definir variables, cursores y condiciones. Se pueden usar varias y se colocan SIEMPRE al principio del bloque y han de crearse en orden.

```
BEGIN  
DECLARE variable1 [DEFAULT valor];  
.....  
END
```

- ▶ Ej:

```
DECLARE x INT;  
DECLARE y INT DEFAULT 0;
```

ESTRUCTURAS DE CONTROL - CONDICIONALES

► IF-THEN-ELSIF-ELSE:

```
IF condicion1 THEN lista_sentencias1  
  [ELSEIF condicion2 THEN lista_sentencias2] ...  
  [ELSE lista_sentenciasn]  
END IF
```

► Ejemplo:

```
BEGIN  
  DECLARE rdo VARCHAR(50);  
  
  IF n1 = n2 THEN SET rdo = 'son iguales';  
  ELSE  
    IF n1 > n2 THEN SET rdo = CONCAT (n1,' es mayor que ', n2);  
    ELSE SET rdo = CONCAT (n1,' es menor que ', n2);  
    END IF;  
  END IF;  
  .....  
  
END;
```

ESTRUCTURAS DE CONTROL - CONDICIONALES

Ejemplo2:

```
BEGIN
  DECLARE signo VARCHAR(20);
  DECLARE rdo VARCHAR(20);

  IF n1 > n2 THEN
    SET signo = '>';
  ELSEIF n1 = n2 THEN
    SET signo = '=';
  ELSE
    SET signo = '<';
  END IF;

  SET rdo = CONCAT(n1, ' ', signo, ' ', n2);

  RETURN rdo;
END ;
```

Ejemplo 3:

```
BEGIN
  DECLARE resultado VARCHAR(50);

  IF nota >= 90 THEN
    SET resultado = 'Excelente';
  ELSIF nota >= 80 THEN
    SET resultado = 'Muy bien';
  ELSIF nota >= 70 THEN
    SET resultado = 'Bien';
  ELSIF nota >= 60 THEN
    SET resultado = 'Aprobado';
  ELSE
    SET resultado = 'Reprobado';
  END IF;

  SELECT resultado AS 'Evaluación';
END;
```

ESTRUCTURAS DE CONTROL - CONDICIONALES

► CASE:

► Uso 1:

```
CASE valor_variable  
  WHEN valor1 THEN lista_sentencias1  
  [WHEN valor2 THEN lista_sentencias2] ...  
  [ELSE lista_sentenciasn]  
END CASE;
```

► Uso 2:

```
CASE  
  WHEN condicion1 THEN lista_sentencias1  
  [WHEN condicion2 THEN lista_sentencias2] ...  
  [ELSE lista_sentenciasn]  
END CASE;
```

Ejemplo:

```
CASE var1  
  WHEN 0 THEN SET var2= 0  
  WHEN 1 THEN SET var2= var1  
  ELSE var2 = (-1)*var1  
END CASE;
```

Ejemplo:

```
CASE  
  WHEN var1 = 0 THEN SET var2 = 0  
  WHEN var1 = 1 THEN SET var2 = var1  
  ELSE var2 = (-1)*var1  
END CASE;
```

ESTRUCTURAS DE CONTROL – BUCLE LOOP

- ▶ **LOOP:** El bucle LOOP repite un bloque de código indefinidamente hasta que se interrumpa manualmente (por ejemplo, con LEAVE).

```
[etiqueta:] LOOP  
    [lista_sentencias]  
    LEAVE etiqueta;  
    [lista_sentencias]  
END LOOP [etiqueta];
```

- ▶ Ejemplo:

```
BEGIN  
    DECLARE cont, total INT;  
    SET cont = 1;  
    SET total = 0;  
  
    bucle: LOOP  
        IF cont > hasta THEN  
            LEAVE bucle;  
        END IF;  
        SET total = total + cont;  
        SET cont = cont + 1;  
    END LOOP bucle;  
END;
```

En este bucle es importante
El uso de **LEAVE**, porque es
lo que establece la condición
de salida. Si lo olvidamos, no
podrá salir del bucle.

ESTRUCTURAS DE CONTROL – BUCLE REPEAT ... UNTIL

- ▶ **REPEAT ... UNTIL:** El bucle REPEAT UNTIL repite un bloque de código hasta que se cumpla una condición lógica, después de ejecutar el código.
- ▶ La condición de salida se evalúa **después** de la primera iteración.
- ▶ Siempre ejecuta **al menos una vez**, incluso si la condición es verdadera al inicio.

```
[etiqueta:] REPEAT  
    lista_sentencias  
UNTIL condicion_de_final  
END REPEAT [etiqueta];
```

- ▶ Ejemplo:

```
BEGIN  
    DECLARE cont, total INT;  
    SET cont = 1;  
    SET total = 0;  
  
    REPEAT  
        SET total = total + cont;  
        SET cont = cont + 1;  
    UNTIL cont > hasta  
    END REPEAT;  
END;
```

Se puede usar cualquier
Bucle sin etiqueta.

ESTRUCTURAS DE CONTROL – BUCLE WHILE ... DO

► **WHILE ... DO:** El bucle WHILE DO repite un bloque de código mientras se cumpla una condición lógica, antes de ejecutar el código.

- La condición de entrada se evalúa **antes** de la primera iteración.
- Si la condición es falsa desde el principio, el bloque no se ejecuta ni una vez.

```
[etiqueta:] WHILE condicion DO  
    lista_sentencias  
END WHILE [etiqueta];
```

► Ejemplo:

```
BEGIN  
    DECLARE cont, total INT;  
    SET cont = 1;  
    SET total = 0;  
  
    WHILE cont <= hasta DO  
        SET total = total + cont;  
        SET cont = cont + 1;  
    END WHILE;  
END ;
```

Se puede usar cualquier
Bucle sin etiqueta.

LEAVE

- ▶ LEAVE es una instrucción en MySQL que permite salir de un bloque o cualquier tipo de bucle etiquetado. Se usa tanto dentro de bucles, así como en bloques BEGIN...END dentro de procedimientos almacenados.
- ▶ Es útil para terminar la ejecución o salir anticipadamente cuando se cumple una condición, sin necesidad de recorrer todo el bucle.

LEAVE etiqueta;

ITERATE

- ▶ ITERATE se utiliza para reiniciar la ejecución de un bucle. Específicamente, se usa dentro de un bucle etiquetado (LOOP, WHILE, REPEAT) para saltarse el resto del código de esa iteración y volver a evaluar la condición de entrada del bucle, comenzando desde el principio de la siguiente iteración.
- ▶ Es una forma de alterar/controlar el flujo de ejecución del bucle de forma manual.
- ▶ Es útil para terminar la ejecución o salir anticipadamente cuando se cumple una condición, sin necesidad de recorrer todo el bucle.

ITERATE etiqueta;

ITERATE

► Ejemplo:

```
BEGIN
  DECLARE i INT DEFAULT 1;

  mi_bucle: LOOP
    IF i > 10 THEN
      LEAVE mi_bucle; -- Salimos del bucle si i > 10
    END IF;

    -- Si el número es par, lo saltamos
    IF i MOD 2 = 0 THEN
      SET i = i + 1;
      ITERATE mi_bucle; -- Reinicia el bucle sin ejecutar el código siguiente
    END IF;

    SELECT CONCAT('Número: ', i); -- Se ejecuta solo si i es impar
    SET i = i + 1;
  END LOOP mi_bucle;
END;
```

PROCEDIMIENTOS Y FUNCIONES

Los procedimientos y las funciones son objetos de la base de datos, por tanto, para crearlos usaremos la sentencia CREATE y para borrarlos usaremos la sentencia DROP.

No podemos alterar el contenido de un procedimiento con ninguna sentencia tipo ALTER, es necesario borrar el procedimiento anterior y volverlo a crear.

```
DELIMITER //
```



```
CREATE PROCEDURE  
nombre_procedimiento()  
BEGIN  
    -- Código del procedimiento  
END //
```



```
DELIMITER ;
```

```
DROP PROCEDURE IF EXISTS nombre_procedimiento;
```

```
DELIMITER //
```



```
CREATE FUNCTION nombre_funcion()  
RETURNS Tipo  
BEGIN  
    -- Código de la función  
END //
```



```
DELIMITER ;
```

```
DROP FUNCTION IF EXISTS nombre_funcion;
```

PROCEDIMIENTOS

- Un procedimiento es un bloque que puede recibir parámetros de entrada, realiza las operaciones deseadas con dichos datos y, en algunos casos guardar ciertos resultados como parámetros de salida (no es obligatorio que devuelvan valores). Para definirlos se usa la palabra reservada PROCEDURE.

```
CREATE PROCEDURE nombre_procedimiento  
    ([ IN | OUT | INOUT ] nombre_param tipo, ...)  
BEGIN  
    cuerpo del programa  
END;
```

- Ej:

```
CREATE PROCEDURE MAXSAL(IN cifra INT, OUT mayorsal FLOAT(10,2))  
BEGIN  
    SET mayorsal =0;  
  
    SELECT MAX(SALARIO)*cifra  
        INTO mayorsal  
        FROM EMPLEADO;  
  
    SELECT mayorsal;  
END;
```

PROCEDIMIENTOS

Pueden no tener parámetros, pero cuando los recibe, pueden ser de tres tipos, y hay que indicarlo:

IN	• Son utilizados para pasar valores al procedimiento desde el código que lo llama. • Permiten que el procedimiento reciba datos desde el exterior, pero no pueden ser modificados dentro del procedimiento (el valor solo se pasa y no cambia).
OUT	• Permiten retornar valores desde el procedimiento al código que lo llama. Estos parámetros no reciben valores al momento de la llamada al procedimiento, pero el procedimiento los usa para devolver datos hacia fuera. • Se utilizan para obtener valores calculados o modificados dentro del procedimiento y hacerlos disponibles para el código que invoca el procedimiento.
IN/OUT	• Son una combinación de IN y OUT. Estos parámetros pueden recibir valores cuando se llama al procedimiento y también pueden devolver valores modificados. Son útiles cuando quieres que el parámetro sea utilizado tanto para enviar datos al procedimiento como para obtener un valor modificado de vuelta. • Permite tanto pasar un valor al procedimiento como obtener un valor modificado dentro del mismo procedimiento.

```
CREATE PROCEDURE nombre_procedimiento(IN p1 INT, OUT p2 VARCHAR(30), OUT p3 DECIMAL(13, 2))
BEGIN
    -- Código del procedimiento
END //
```

PROCEDIMIENTOS

En MySQL, los procedimientos almacenados se ejecutan usando la sentencia **CALL**. Dependiendo del tipo de parámetro que tenga el procedimiento (IN, OUT o INOUT), la forma de llamar puede variar.

Tipo de parámetro	Uso	Qué se le pasa en el CALL
IN	Se usa para pasar un valor de entrada al procedimiento (solo lectura dentro del procedimiento).	Se envía un valor directamente
OUT	Se usa para devolver un valor desde el procedimiento (no recibe valores de entrada).	Se pasa una variable que se modificará.
INOUT	Se usa para recibir y devolver un valor (el procedimiento puede modificar su valor).	Se pasa una variable que puede ser modificada.

PROCEDIMIENTOS

Ejemplos:

PROCEDURE	LLAMADA
<pre>CREATE PROCEDURE obtener_nombre_usuario(IN user_id INT) BEGIN SELECT nombre FROM usuarios WHERE id = user_id; END \$\$</pre>	<pre>CALL obtener_nombre_usuario(3);</pre>
<pre>CREATE PROCEDURE total_pedidos_usuario(IN user_id INT, OUT total_pedidos INT) BEGIN SELECT COUNT(*) INTO total_pedidos FROM pedidos WHERE usuario_id = user_id; END \$\$</pre>	<pre>-- Declaro una variable que recibirá el resultado SET @total = 0; -- Llamo al procedimiento CALL total_pedidos_usuario(3, @total); -- Muestro el valor devuelto SELECT @total;</pre>
<pre>CREATE PROCEDURE duplicar_valor(INOUT num INT) BEGIN SET num = num * 2; END \$\$</pre>	<pre>-- Definir una variable inicial SET @valor = 5; -- Llamo al procedimiento CALL duplicar_valor(@valor); -- Verifico que @valor es 10 SELECT @valor;</pre>

FUNCIONES

- ▶ Una función es muy similar a un procedimiento. Puede recibir sólo parámetros de entrada y realizar operaciones con dichos datos. Lo que distingue a una función de un procedimiento es que la función siempre devuelve (sección RETURNS) algún valor. Para definirlas se usa la palabra reservada FUNCTION.
- ▶ Como solamente puede recibir parámetros de tipo IN, no es necesario indicarlo en la definición de la función.
- ▶ Sí es necesario indicar el TIPO del valor que devuelve.
- ▶ La palabra RETURN siempre ha de estar presente en el bloque BEGIN END acompañada del valor que devuelve la función.
- ▶ Posibles valores de [opciones]:
 - ▶ DETERMINISTIC / NOT DETERMINISTIC: indica si al ejecutar la función con las mismas opciones de entrada varias veces devuelve o no el mismo valor.
 - ▶ CONTAINS SQL: contiene código SQL pero no tiene sentencias que lean/escriben datos.
 - ▶ NO SQL: contiene instrucciones de código NO SQL
 - ▶ READS SQL DATA: contiene sentencias que lean (consultas SQL) pero no que escriban datos.
 - ▶ MODIFIES SQL DATA : contiene sentencias SQL que modifican datos en las tablas (INSERT/UPDATE/DELETE)

```
CREATE FUNCTION nombre_funcion  
  (nombre_param tipo, ...) RETURNS tipo [opciones]  
BEGIN  
  cuerpo del programa  
  RETURN resultado;  
END;
```

FUNCIONES

► Ej:

```
CREATE FUNCTION DEPART(empleado VARCHAR(15))  
RETURNS CHAR(5) DETERMINISTIC  
READS SQL DATA  
BEGIN  
  
    DECLARE departamento CHAR(5);  
  
    SELECT DEPTNO  
        INTO departamento  
        FROM EMPLEADO  
        WHERE codemp = empleado;  
  
    RETURN departamento;  
  
END;
```

Por motivos relacionados con la seguridad en el procesamiento, es necesario al menos definir alguna de estas opciones, como DETERMINISTIC, READS SQL DATA o NO SQL.

O bien, podemos evitar este requerimiento haciendo la ejecución algo menos segura usando:

```
SET GLOBAL log_bin_trust_function_creators = 1;
```

FUNCIONES

En MySQL, las funciones almacenadas son similares a los procedimientos almacenados, pero tienen una diferencia clave: las funciones devuelven un valor y se pueden usar dentro de una consulta SELECT, en WHERE, ORDER BY, etc.

A diferencia de los procedimientos, las funciones se llaman dentro de una consulta SQL usando SELECT o dentro de otra sentencia.

FUNCIÓN	LLAMADAS
<pre>CREATE FUNCTION calcular_descuento(precio DECIMAL(10,2), descuento DECIMAL(5,2)) RETURNS DECIMAL(10,2) DETERMINISTIC BEGIN DECLARE precio_final DECIMAL(10,2); SET precio = precio - (precio * descuento / 100); RETURN precio; END \$\$</pre>	<pre>SELECT calcular_descuento(100, 10) AS Precio_Rebajado;</pre>
	<pre>--También la puedo asignar a una variable: SET @precio_rebajado = calcular_descuento(100, 15); --Verifico que el resultado es correcto SELECT @precio_final; -- Devolverá 85.00</pre>

Listar procedimientos / funciones de mi DB

- ▶ Puedo usar:

```
SHOW PROCEDURE STATUS WHERE db = '<database_name>';  
SHOW FUNCTION STATUS WHERE db = '<database_name>';
```

- ▶ También:

```
SELECT  
    routine_name  
FROM  
    information_schema.routines  
WHERE  
    routine_type = 'PROCEDURE'  
    AND routine_schema = '<database_name>';
```

```
SELECT  
    routine_name  
FROM  
    information_schema.routines  
WHERE  
    routine_type = 'FUNCTION'  
    AND routine_schema = '<database_name>';
```

Preguntas

